

Z-transfer Functions and FIR Filter Implementation

Lab 1 · Digital Signal Processing Laboratory

Objectives

1. To read off the difference equation of an FIR (finite-impulse-response) system directly from its coefficients, and vice versa.
2. To compute and plot the frequency response of FIR systems and to carry out digital filtering of a test signal, confirming the result in the frequency domain.

Part A: A Simple FIR System

Consider the FIR system with numerator coefficients $b = [0.2 \ 0.3 \ 0.2]$.

(a) Transfer function

$$H(z) = 0.2 + 0.3z^{-1} + 0.2z^{-2}$$

(b) Difference equation

$$y(n) = 0.2x(n) + 0.3x(n-1) + 0.2x(n-2)$$

(c) Digital filtering

The difference equation was coded as an explicit loop over the input `sample = [1:1:10]` and then compared with MATLAB's built-in `filter()`. The two agree term for term:

n	1	2	3	4	5	6	7	8	9	10
y	0.2	0.7	1.4	2.1	2.8	3.5	4.2	4.9	5.6	6.3

Table 1: Output $y(3:\text{length}(x))$ for `sample = 1 to 10`.

Is $y(3:\text{length}(\text{sample}))$ equal to yy ? Yes. The hand-written difference-equation loop and the library `filter()` perform the same convolution, so the two output vectors come out identical.

The Two 25-tap FIR Systems

Two 25-tap coefficient sets are supplied: a low-pass set bLP and a band-pass set bBP (both listed inside the program). Taking the first system (bLP) as the worked example:

Transfer function (from the z-transform)

$$H(z) = \sum_{k=0}^{24} bLP(k+1)z^{-k} = -0.0012 - 0.0025z^{-1} - \dots + 0.1500z^{-12} + \dots - 0.0012z^{-24}$$

Difference equation

$$y(n) = \sum_{k=0}^{24} bLP(k+1)x(n-k)$$

An FIR filter carries no feedback (its denominator is 1), so each output sample is simply a weighted sum of the current input and the 24 preceding inputs, the 25 weights being the coefficients in *bLP*.

MATLAB Program

```
clc; clear; close all;

% =====
% LAB 1 : Z-transfer Functions, Difference Equations & FIR Filters
% =====

matric = 'output';

%% ===== PART A : simple FIR =====
% FIR: b = [0.2 0.3 0.2]
% H(z) = 0.2 + 0.3 z^-1 + 0.2 z^-2
% y(n) = 0.2 x(n) + 0.3 x(n-1) + 0.2 x(n-2)

sample = [1:1:10];
x = [0 0 sample]; % zero-pad for the two initial delays
y = zeros(1,length(x));
for n = 3:1:length(x)
    y(n) = 0.2*x(n) + 0.3*x(n-1) + 0.2*x(n-2); % difference equation
end
y_manual = y(3:1:length(x));

yy = filter([0.2 0.3 0.2],1,sample); % built-in verification

fprintf('--- PART A: FIR b = [0.2 0.3 0.2] ---\n');
disp('Manual loop y(3:end) ='); disp(y_manual);
disp('filter() yy ='); disp(yy);
if isequal(round(y_manual,10), round(yy,10))
    fprintf('Result: y(3:length(x)) IS the same as yy.\n\n');
else
    fprintf('Result: y(3:length(x)) is NOT the same as yy.\n\n');
end

%% ===== FIR system coefficients =====
bLP = [ -0.0012 -0.0025 -0.0045 -0.0068 -0.0073 -0.0030 0.0089 ...
        0.0297 0.0583 0.0907 0.1208 0.1422 0.1500 0.1422 ...
        0.1208 0.0907 0.0583 0.0297 0.0089 -0.0030 -0.0073 ...
        -0.0068 -0.0045 -0.0025 -0.0012 ];

bBP = [ 0.0004 -0.0017 -0.0064 -0.0076 0.0073 0.0363 0.0458 ...
        0.0000 -0.0802 -0.1134 -0.0419 0.0860 0.1500 0.0860 ...
        -0.0419 -0.1134 -0.0802 0.0000 0.0458 0.0363 0.0073 ...
        -0.0076 -0.0064 -0.0017 0.0004 ];

% For FIR system 1 (bLP):
% H(z) = sum_{k=0}^{24} bLP(k+1) * z^-k
% y(n) = sum_{k=0}^{24} bLP(k+1) * x(n-k)

%% ===== Frequency responses (freqz) =====
[H1,f1] = freqz(bLP,1,512,8000);
```

```

figure('Name','FIR 1 (LP) response','Position',[100 100 700 500]);
subplot(2,1,1); plot(f1,20*log10(abs(H1)));
title('FIR System 1 (bLP) - Magnitude Response');
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)'); grid on;
subplot(2,1,2); plot(f1,angle(H1)*180/pi);
title('FIR System 1 (bLP) - Phase Response');
xlabel('Frequency (Hz)'); ylabel('Phase (deg)'); grid on;
print(gcf,[matric '_FIR1_LP_freqz'],'-dpng','-r200');

[H2,f2] = freqz(bBP,1,512,8000);
figure('Name','FIR 2 (BP) response','Position',[100 100 700 500]);
subplot(2,1,1); plot(f2,20*log10(abs(H2)));
title('FIR System 2 (bBP) - Magnitude Response');
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)'); grid on;
subplot(2,1,2); plot(f2,angle(H2)*180/pi);
title('FIR System 2 (bBP) - Phase Response');
xlabel('Frequency (Hz)'); ylabel('Phase (deg)'); grid on;
print(gcf,[matric '_FIR2_BP_freqz'],'-dpng','-r200');

%% ===== Sum of sinusoids (1 s, 8 kHz) =====
fs = 8000;
dur = 1;
t = 0:1/fs:dur-1/fs; % 8000 samples -> 1 Hz FFT resolution
xin = 5*cos(2*pi*500*t) ...
      + 5*cos(2*pi*1200*t + 0.25*pi) ...
      + 5*cos(2*pi*1800*t + 0.5*pi);

%% ===== Digital filtering + spectra =====
yLP = filter(bLP,1,xin); % through FIR system 1
yBP = filter(bBP,1,xin); % through FIR system 2

N = length(xin); % = 8000 -> resolution fs/N = 1 Hz
f = (0:N/2)*(fs/N);
MX = abs(fft(xin,N)); MX = MX(1:N/2+1)/N*2;
MLP = abs(fft(yLP,N)); MLP = MLP(1:N/2+1)/N*2;
MBP = abs(fft(yBP,N)); MBP = MBP(1:N/2+1)/N*2;

figure('Name','FIR spectra','Position',[100 100 700 700]);
subplot(3,1,1); plot(f,MX); xlim([0 4000]);
title('Input Signal Spectrum x(t)'); xlabel('Frequency (Hz)'); ylabel('Magnitude'); grid on;
subplot(3,1,2); plot(f,MLP); xlim([0 4000]);
title('Output Spectrum - FIR System 1 (Lowpass bLP)'); xlabel('Frequency (Hz)');
ylabel('Magnitude'); grid on;
subplot(3,1,3); plot(f,MBP); xlim([0 4000]);
title('Output Spectrum - FIR System 2 (Bandpass bBP)'); xlabel('Frequency (Hz)');
ylabel('Magnitude'); grid on;
print(gcf,[matric '_FIR_Spectra'],'-dpng','-r200');

% Report the magnitude at the three input frequencies for each output
peakAt = @(M,fq) M(round(fq)+1); % 1 Hz resolution -> bin index = freq+1
fprintf('--- Output magnitudes at the input frequencies ---\n');
fprintf('      500 Hz   1200 Hz   1800 Hz\n');
fprintf('Input   : %7.3f %7.3f %7.3f\n', peakAt(MX,500), peakAt(MX,1200),
peakAt(MX,1800));
fprintf('bLP out : %7.3f %7.3f %7.3f\n', peakAt(MLP,500), peakAt(MLP,1200),
peakAt(MLP,1800));
fprintf('bBP out : %7.3f %7.3f %7.3f\n', peakAt(MBP,500), peakAt(MBP,1200),
peakAt(MBP,1800));

%% ===== Listen & compare (uncomment) =====
% sound(xin/max(abs(xin)), fs); pause(dur+0.5);
% sound(yLP/max(abs(yLP)), fs); pause(dur+0.5);

```

```
% sound(yBP/max(abs(yBP)), fs); pause(dur+0.5);  
  
disp(' ');  
disp('Done. All figures saved with the output prefix.');
```

Results and Discussion

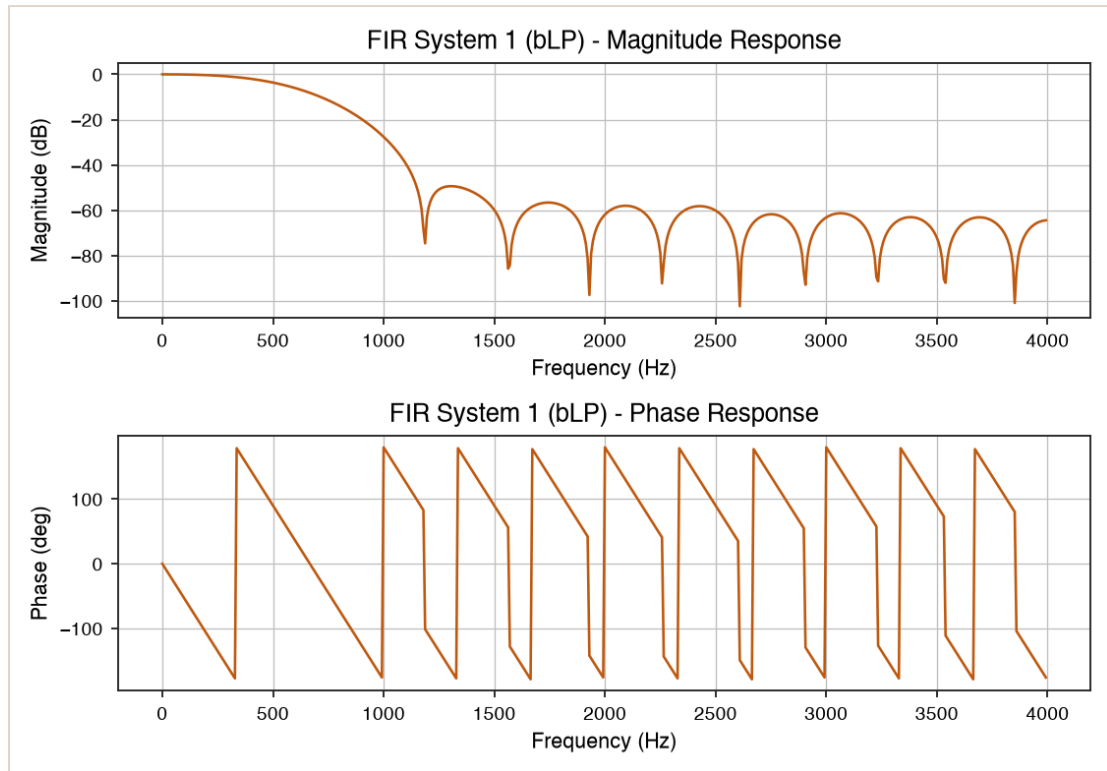


Figure 1: Frequency response of FIR System 1 (*bLP*). The gain holds near 0 dB at low frequencies and then drops away quickly past roughly 1 kHz, i.e. a low-pass shape. The phase runs straight through the passband, as expected of a symmetric FIR filter.

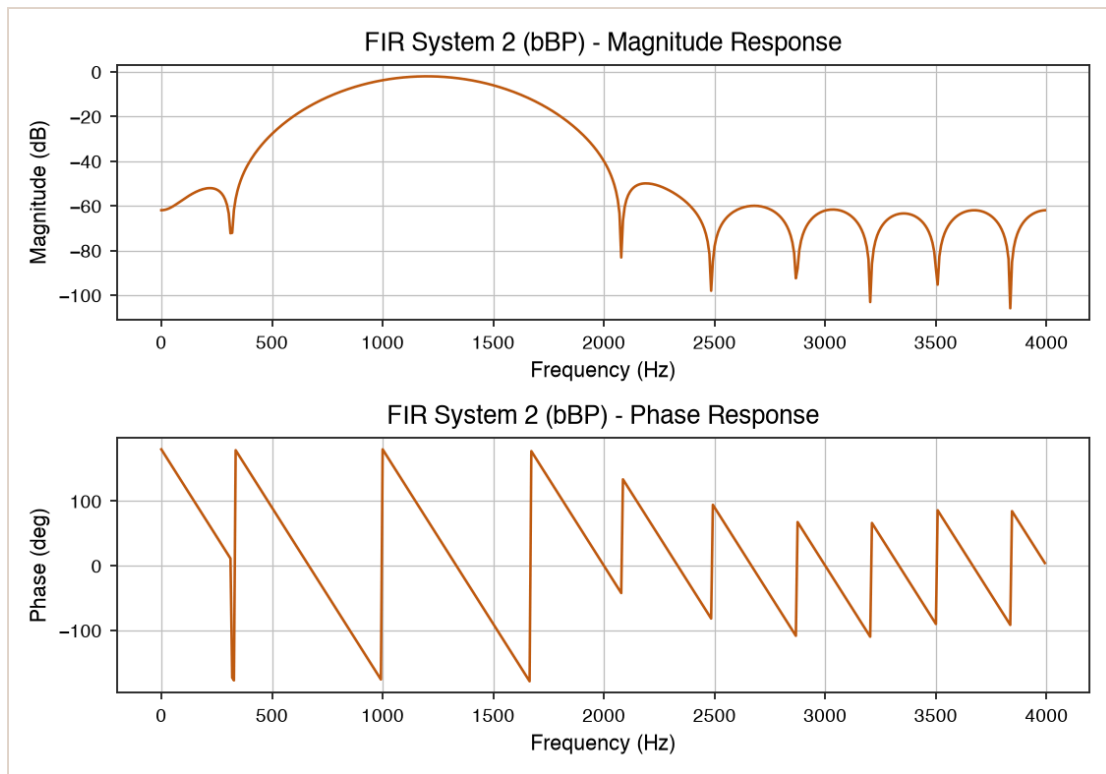


Figure 2: Frequency response of FIR System 2 (*bBP*). The gain rises to a maximum near 0 dB around 1200 Hz and is pushed down on either side, i.e. a band-pass shape centred on the middle of the band.

A one-second, three-tone test signal was generated at $f_s = 8$ kHz:

$$x(t) = 5\cos(2\pi 500t) + 5\cos(2\pi 1200t + 0.25\pi) + 5\cos(2\pi 1800t + 0.5\pi)$$

With a record length of $N = f_s = 8000$ samples, the spectra land on a 1 Hz grid. The signal was passed through each FIR system in turn.

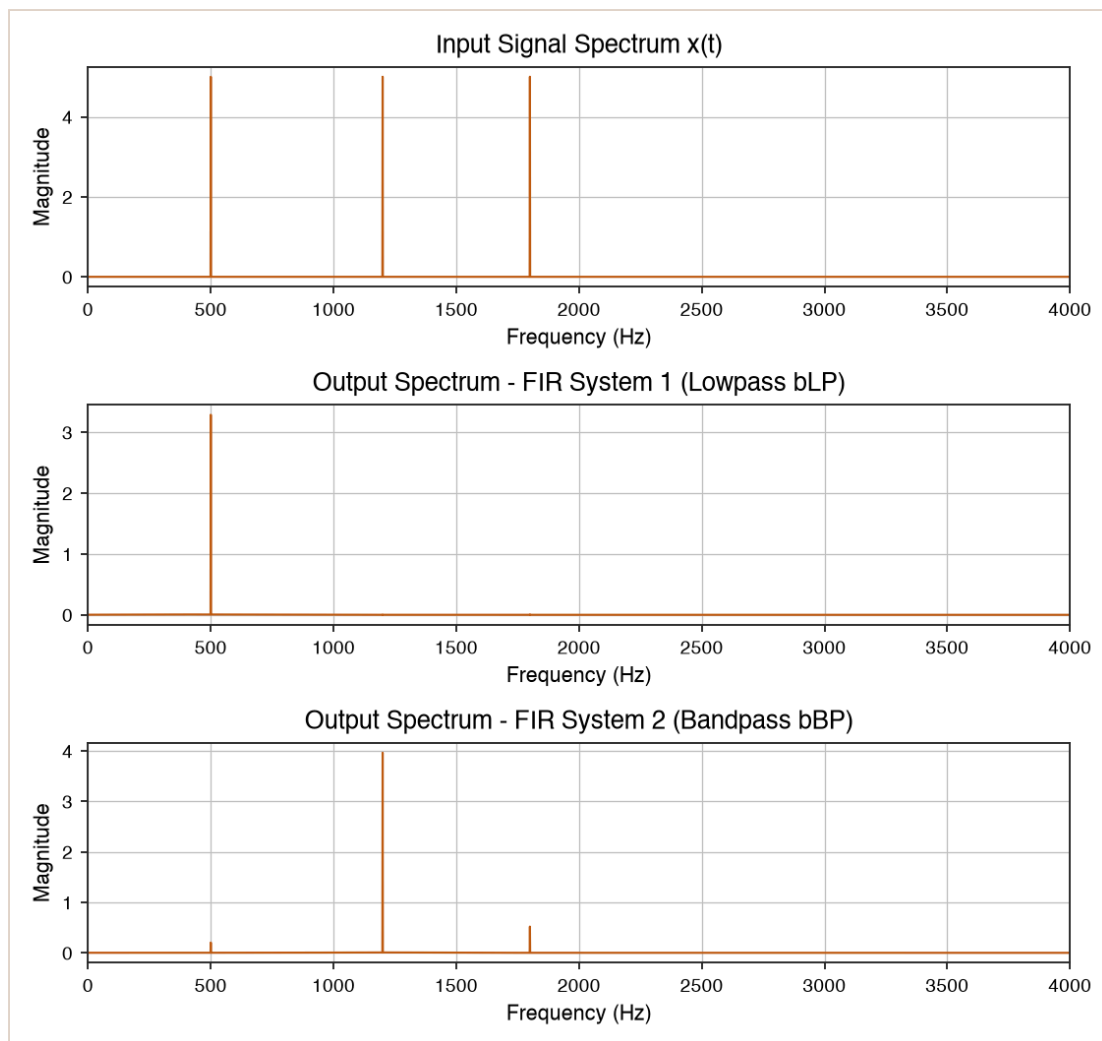


Figure 3: The input spectrum (three equal lines at 500, 1200 and 1800 Hz) and the two output spectra. The low-pass system keeps only the 500 Hz tone; the band-pass system keeps only the 1200 Hz tone.

	500 Hz	1200 Hz	1800 Hz
Input	5.000	5.000	5.000
bLP output (low-pass)	3.288	0.005	0.007
bBP output (band-pass)	0.204	3.963	0.519

Table 2: Output magnitude at each input frequency (input amplitude = 5).

Which components survive at the output? After the low-pass system (*bLP*) only the 500 Hz line remains, the 1200 and 1800 Hz tones being almost entirely stripped out. After the band-pass system (*bBP*) the 1200 Hz line dominates, 1800 Hz is heavily reduced and 500 Hz is all but gone. On playback the low-pass output is a low hum carrying only the lowest tone, while the band-pass output isolates the middle tone.

Conclusion

The transfer function and difference equation of an FIR system follow straight from its numerator coefficients, and the hand-coded difference equation reproduced MATLAB's `filter()` output exactly. The computed responses marked *bLP* as a low-pass filter and *bBP* as a band-pass filter, and the three-tone filtering test bore this out, since each filter retained only the tone lying inside its passband and rejected the rest.